# Übungsblatt 6: Das Steuerwerk

**Mikrocomputertechnik 1**

**Musterlösung**

## Aufgabe 1: Was soll der Datenpfad tun? (8 Punkte)

Single-Cycle-Kursmodell aus Einheit 5 und 6. Jeder Befehl startet unabhängig mit t1 = 12, t2 = 30, s0 = `0x2000`; das Wort an `0x2004` ist 7. Die erste Zeile ist ein Beispiel, „—" bedeutet keine Wirkung.

1. Ergänzen Sie die Tabelle. (4 P)

| Befehl | ALU-Eingang B | ALU-Ergebnis | Datenspeicher | Register danach |
|---|---|---|---|---|
| `add t3, t1, t2` | RD2 = 30 | 42 | — | t3 = 42 |
| `lw t3, 4(s0)` | | | | |
| `sw t2, 8(s0)` | | | | |
| `beq t1, t2, ziel` | | | | |
| `addi t3, t1, -2` | | | | |

2. Bei zwei Befehlen ist es gleichgültig, ob das ALU-Ergebnis oder das Speicherwort zum Register-Schreibport läuft. Welche sind es, und warum? (2 P)
3. addi gleicht bei ALU-Eingang B einem Befehl der Tabelle, beim Register-Schreibwert einem anderen. Welchen jeweils? Welche Werte folgen daraus für ALUSrc und MemtoReg? (2 P)

**Musterlösung**

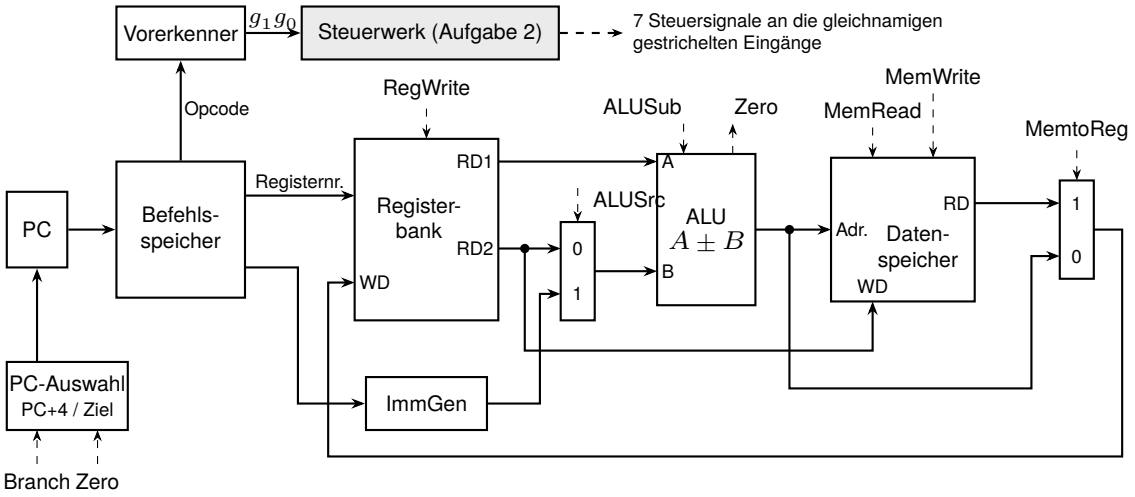
| Befehl | ALU-Eingang B | ALU-Ergebnis | Datenspeicher | Register danach |
|---|---|---|---|---|
| `lw t3, 4(s0)` | Immediate 4 | Adresse `0x2004` | liest 7 | t3 = 7 |
| `sw t2, 8(s0)` | Immediate 8 | Adresse `0x2008` | schreibt 30 an `0x2008` | — |
| `beq t1, t2, ziel` | RD2 = 30 | $12 - 30 = -18$, Zero = 0 | — | — (PC + 4) |
| `addi t3, t1, -2` | Immediate -2 | 10 | — | t3 = 10 |

1. Je Zeile 1 P.

2. sw und beq (1 P): Beide schreiben kein Register, RegWrite ist null. Die Auswahl ist daher ein Don't Care X (1 P). Die Store-Daten kommen bei sw direkt aus RD2.
3. Eingang B wie lw (Immediate), Schreibwert wie add (ALU-Ergebnis) (1 P). Daraus folgen ALUSrc = 1 und MemtoReg = 0 (1 P).

## Aufgabe 2: Ein Steuerwerk für den Mini-Datenpfad bauen (16 Punkte)

Der Mini-Datenpfad kennt nur `add`, `beq`, `lw` und `sw`. Ein Vorerkenner hat den Befehl bereits als gültig erkannt und liefert den Gruppencode $g_1 g_0$: 00 = add, 01 = beq, 10 = lw, 11 = sw. Dieser Code ist eine Annahme des Mini-Modells, kein RISC-V-Feld. Die ALU kann nur addieren und subtrahieren; statt ALUOp genügt deshalb das Signal ALUSub. Gestrichelte Pfeile sind Steuersignale, gleichnamige Anschlüsse sind verbunden.




| Signal | wirkt auf | 0 bedeutet | 1 bedeutet |
|---|---|---|---|
| ALUSrc | MUX vor ALU-Eingang B | RD2 | Immediate |
| ALUSub | ALU | $A + B$ | $A - B$ |
| MemtoReg | Rückschreib-MUX | ALU-Ergebnis | Speicherwort |
| RegWrite | Registerbank | kein Schreiben | Schreiben an der Flanke |
| MemRead | Datenspeicher | kein Lesen | Lesen |
| MemWrite | Datenspeicher | kein Schreiben | Schreiben an der Flanke |
| Branch | PC-Auswahl | immer PC + 4 | Ziel, falls Zero = 1 |

1. Vervollständigen Sie die Wahrheitstabelle. Verwenden Sie X nur, wo die Auswahl nachweislich keine Wirkung hat. (6 P)

| $g_1 g_0$ | Befehl | ALUSrc | ALUSub | MemtoReg | RegWrite | MemRead | MemWrite | Branch |
|---|---|---|---|---|---|---|---|---|
| 00 | add | 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 01 | beq | | | | | | | |
| 10 | lw | | | | | | | |
| 11 | sw | | | | | | | |

2. Geben Sie jedes Signal als Formel in $g_1$ und $g_0$ an. Beispiel: MemWrite $= g_1 \wedge g_0$, denn nur sw schreibt in den Speicher. Tipp: Suchen Sie die Zeilen mit einer Eins. Manche Signale sind direkt ein Eingangsbit oder dessen Negation. (6 P)

**Musterlösung 1 und 2**

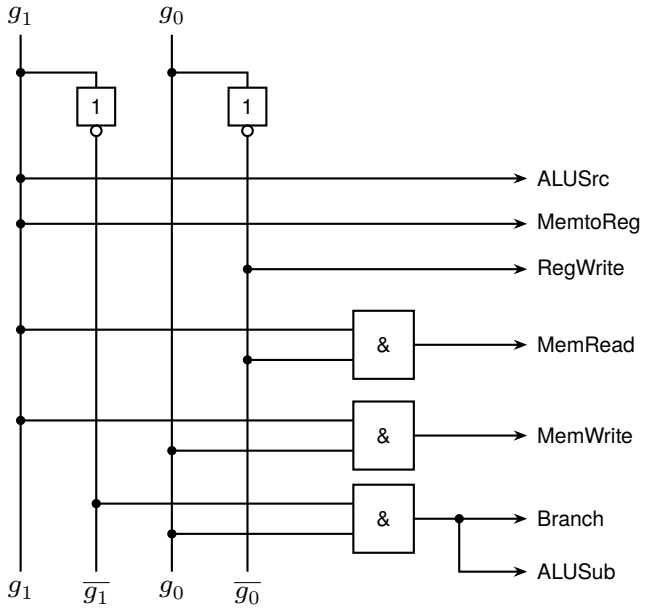
| $g_1 g_0$ | Befehl | ALUSrc | ALUSub | MemtoReg | RegWrite | MemRead | MemWrite | Branch |
|---|---|---|---|---|---|---|---|---|
| 01 | beq | 0 | 1 | X | 0 | 0 | 0 | 1 |
| 10 | lw | 1 | 0 | 1 | 1 | 1 | 0 | 0 |
| 11 | sw | 1 | 0 | X | 0 | 0 | 1 | 0 |

1. Je Zeile 2 P, anteilig. Für MemtoReg bei beq und sw ist statt X auch ein konkreter Wert richtig, wenn die Begründung RegWrite = 0 nennt.
2. Je Signal 1 P:
   - ALUSrc $= g_1$: lw und sw sind genau die Codes mit $g_1 = 1$.
   - ALUSub $= \overline{g_1} \wedge g_0$: nur beq.
   - MemtoReg $= g_1$ (X bei sw als 1, bei beq als 0 gewählt) oder ebenso richtig $g_1 \wedge \overline{g_0}$.
   - RegWrite $= \overline{g_0}$: add und lw sind genau die Codes mit $g_0 = 0$.
   - MemRead $= g_1 \wedge \overline{g_0}$: nur lw.
   - Branch $= \overline{g_1} \wedge g_0$: nur beq, also gleich ALUSub.

3. Zeichnen Sie das Steuerwerk mit Negationen (NICHT) und UND-Gattern in den Kasten. Welche Ausgänge brauchen kein eigenes UND-Gatter? (2 P)

4. Die PC-Auswahl bildet PCSrc $=$ Branch $\wedge$ Zero. Bestimmen Sie Branch, Zero und PCSrc für `beq` mit den Werten 7 und 7, `beq` mit 7 und 9 sowie `add t0, x0, x0`. (2 P)

**Musterlösung 3 und 4**




3. Zwei Negationen und drei UND-Gatter; Punkte markieren Verbindungen, Kreuzungen ohne Punkt sind nicht verbunden (1 P). Kein eigenes UND-Gatter brauchen ALUSrc und MemtoReg (direkt $g_1$), RegWrite (nur $\overline{g_0}$) und ALUSub (gleich Branch) (1 P).

| Fall | Branch | Zero | PCSrc |
|---|---|---|---|
| beq, 7 und 7 | 1 | 1 | 1 |
| beq, 7 und 9 | 1 | 0 | 0 |
| `add t0, x0, x0` | 0 | 1 | 0 |

4. Je Fall 2/3 P. Branch folgt aus der Befehlsart, Zero aus dem ALU-Ergebnis.

## Aufgabe 3: Ein Programm durch den Mini-Datenpfad verfolgen (10 Punkte)

Der Mini-Datenpfad aus Aufgabe 2 arbeitet mit Ihrem Steuerwerk. Anfang: s0 = `0x2000`, t1 bis t4 = 0, t5 = 99. Die Datenwörter ab `0x2000` lauten 12, 30, 0, 42 (Wert A, Wert B, Summe, Sollwert).

```
start:         lw   t1, 0(s0)         # 0x100  Wert A
               lw   t2, 4(s0)         # 0x104  Wert B
               add  t3, t1, t2        # 0x108  Summe
               sw   t3, 8(s0)         # 0x10C  Summe ablegen
               lw   t4, 12(s0)        # 0x110  Sollwert
               beq  t3, t4, treffer   # 0x114
kein_treffer: add  t5, x0, x0        # 0x118
               beq  x0, x0, fertig    # 0x11C
treffer:      add  t5, t3, x0        # 0x120
fertig:        beq  x0, x0, fertig    # 0x124
```

1. Füllen Sie die Tabelle bis zum Erreichen von `fertig` aus. Freigaben sind RegWrite, MemRead, MemWrite und Branch; tragen Sie diejenigen ein, die eins sind. Die letzte Zeile bestimmen Sie selbst. (7 P)

| PC | Befehl | $g_1 g_0$ | ALU-Ergebnis | Freigaben = 1 | Wirkung | nächster PC |
|---|---|---|---|---|---|---|
| `0x100` | `lw t1, 0(s0)` | | | | | |
| `0x104` | `lw t2, 4(s0)` | | | | | |
| `0x108` | `add t3, t1, t2` | | | | | |
| `0x10C` | `sw t3, 8(s0)` | | | | | |
| `0x110` | `lw t4, 12(s0)` | | | | | |
| `0x114` | `beq t3, t4, treffer` | | | | | |

2. Welche Befehle werden übersprungen? Woran erkennen Sie das in der beq-Zeile? (1 P)
3. Geben Sie t1 bis t5 und die vier Datenwörter bei `fertig` an. (2 P)

**Musterlösung**

| PC | Befehl | $g_1 g_0$ | ALU-Ergebnis | Freigaben = 1 | Wirkung | nächster PC |
|---|---|---|---|---|---|---|
| `0x100` | `lw t1, 0(s0)` | 10 | `0x2000` | RegWrite, MemRead | t1 = 12 | `0x104` |
| `0x104` | `lw t2, 4(s0)` | 10 | `0x2004` | RegWrite, MemRead | t2 = 30 | `0x108` |
| `0x108` | `add t3, t1, t2` | 00 | 42 | RegWrite | t3 = 42 | `0x10C` |
| `0x10C` | `sw t3, 8(s0)` | 11 | `0x2008` | MemWrite | Wort an `0x2008` = 42 | `0x110` |
| `0x110` | `lw t4, 12(s0)` | 10 | `0x200C` | RegWrite, MemRead | t4 = 42 | `0x114` |

| PC | Befehl | $g_1 g_0$ | ALU-Ergebnis | Freigaben = 1 | Wirkung | nächster PC |
|---|---|---|---|---|---|---|
| `0x114` | `beq t3, t4, treffer` | 01 | 0, Zero = 1 | Branch | — | `0x120` |
| `0x120` | `add t5, t3, x0` | 00 | 42 | RegWrite | t5 = 42 | `0x124` |

1. Je Zeile 1 P.
2. Die beiden Befehle bei `kein_treffer` (`0x118`, `0x11C`): Branch = 1 und Zero = 1 ergeben PCSrc = 1, der PC springt nach `0x120`. 1 P.
3. t1 = 12, t2 = 30, t3 = 42, t4 = 42, t5 = 42 (1 P); Datenwörter 12, 30, 42, 42 (1 P).

## Aufgabe 4: Fehlersuche im Steuerwerk (8 Punkte)

Ein Team hat das Steuerwerk aus Aufgabe 2 aufgebaut und startet das Programm aus Aufgabe 3 mit demselben Anfangszustand. Bei `fertig` steht in t1 der Wert `0x2000` statt 12.

1. Welcher Wert wurde offenbar statt des Speicherworts nach t1 geschrieben? Welches Steuersignal ist deshalb verdächtig? (2 P)
2. Tatsächlich ist MemtoReg direkt mit $g_0$ verbunden; alle anderen Signale sind richtig. Vergleichen Sie MemtoReg $= g_0$ für alle vier Gruppencodes mit Ihrer Tabelle. Bei welchem Befehl wirkt der Fehler, bei welchen ist er harmlos? (2 P)
3. Verfolgen Sie das Programm mit diesem Fehler. Geben Sie t1 bis t5, das Wort an `0x2008` und den genommenen Pfad (`treffer` oder `kein_treffer`) an. (3 P)
4. Geben Sie eine korrekte Formel für MemtoReg an. (1 P)

**Musterlösung**

1. Die ALU-Adresse `0x2000` des ersten Loads (1 P). Verdächtig ist MemtoReg, die Rückschreibauswahl (1 P). RegWrite war offensichtlich eins, sonst wäre t1 null geblieben.
2. add: $g_0 = 0$, richtig. lw: $g_0 = 0$ statt 1, das ALU-Ergebnis wird geschrieben; hier wirkt der Fehler (1 P). beq und sw: 1 statt X, harmlos wegen RegWrite = 0 (1 P).
3. t1 = `0x2000`, t2 = `0x2004`, t3 = `0x4004`, t4 = `0x200C` (2 P, anteilig). Das Wort an `0x2008` wird `0x4004`, denn Store-Adresse und Store-Daten aus RD2 sind unverändert richtig. beq vergleicht `0x4004` mit `0x200C`: kein Sprung, Pfad `kein_treffer`, t5 = 0 (zusammen 1 P).
4. MemtoReg $= g_1$ oder MemtoReg $= g_1 \wedge \overline{g_0}$. 1 P.

## Aufgabe 5: Ausbaustufe mit vier Rechenbefehlen (8 Punkte)

Gruppencode 00 steht nun für die vier Registerbefehle `add`, `sub`, `and` und `or`. Die Hauptsteuerung liefert statt ALUSub die Kategorie ALUOp; die ALU-Steuerung bildet daraus wie in Einheit 6 das Kommando ALUControl.

| ALUOp | Bedeutung |
|---|---|
| 00 | Addition |
| 01 | Subtraktion |
| 10 | Operation aus funct3 und funct7 |

| Befehl | funct3 | funct7 | ALUControl |
|---|---|---|---|
| add | 000 | 0000000 | 0010 |
| sub | 000 | 0100000 | 0110 |
| and | 111 | 0000000 | 0000 |
| or | 110 | 0000000 | 0001 |

1. Geben Sie ALUOp für die Gruppencodes 00, 01, 10 und 11 an. (2 P)
2. t1 = 12 = $1100_2$, t2 = 10 = $1010_2$, s0 = `0x2000`. Bestimmen Sie ALUOp, ALUControl und ALU-Ergebnis. (5 P)

| Befehl | ALUOp | ALUControl | ALU-Ergebnis |
|---|---|---|---|
| `add t0, t1, t2` | | | |
| `sub t0, t1, t2` | | | |
| `and t0, t1, t2` | | | |
| `or t0, t1, t2` | | | |
| `lw t0, 8(s0)` | | | |

3. Warum kann die Hauptsteuerung die Rechenoperation jetzt nicht mehr allein aus $g_1 g_0$ festlegen? (1 P)

**Musterlösung**

1. 00 → 10, 01 → 01, 10 → 00, 11 → 00. Je 0,5 P.

| Befehl | ALUOp | ALUControl | ALU-Ergebnis |
|---|---|---|---|
| `add t0, t1, t2` | 10 | 0010 | 22 |
| `sub t0, t1, t2` | 10 | 0110 | 2 |
| `and t0, t1, t2` | 10 | 0000 | 8 = $1000_2$ |
| `or t0, t1, t2` | 10 | 0001 | 14 = $1110_2$ |
| `lw t0, 8(s0)` | 00 | 0010 | Adresse `0x2008` |

2. Je Zeile 1 P.
3. Code 00 umfasst vier Operationen mit demselben Datenweg; erst funct3 und funct7 unterscheiden sie. 1 P.

## Aufgabe 6: Das Programm in Ripes prüfen

Ab `start` führt das Programm genau den Ablauf aus Aufgabe 3 aus. Davor lädt `la` die Adresse des Datenbereichs `werte` nach s0, und `li` setzt t5 auf 99.

```
.data
.align 2
werte: .word 12, 30, 0, 42
.text
.globl main
main:
    la s0, werte
    li t5, 99
start:
    lw t1, 0(s0)
    lw t2, 4(s0)
    add t3, t1, t2
    sw t3, 8(s0)
    lw t4, 12(s0)
    beq t3, t4, treffer
kein_treffer:
    add t5, x0, x0
    beq x0, x0, fertig
treffer:
    add t5, t3, x0
fertig:
    beq x0, x0, fertig
```

1. Geben Sie das Programm im Editor von Ripes ein. Wählen Sie im Tab *Processor* den Single-Cycle-Prozessor (RV32I, ohne C) mit dem Layout *Extended*; nur dieses zeigt das Steuerwerk. Protokollieren Sie Version und Modell. Lesen Sie die tatsächliche Adresse B von `werte` ab; sie ersetzt `0x2000` aus Aufgabe 3.
2. Halten Sie vor jedem Befehl ab `start` an. Notieren Sie vor der Clock die Schreibfreigaben von Registerbank und Datenspeicher sowie die Auswahl vor ALU-Eingang B und am Rückschreib-MUX. Vergleichen Sie mit Ihrer Tabelle aus Aufgabe 2. Prüfen Sie nach der Clock Register, Speicherwort und PC.
3. Bestätigen Sie bei `fertig` t1 bis t5 und die vier Wörter ab B.
4. Ändern Sie im Datenabschnitt den Sollwert von 42 auf 41. Sagen Sie Pfad und t5 voraus, bevor Sie das Programm neu starten, und prüfen Sie dann.
5. Beim `sw` zeigt Ripes am Rückschreib-MUX einen konkreten Auswahlwert, in Ihrer Tabelle steht X. Warum ist das kein Widerspruch?

Die Initialisierung vor `start` und die Schleife bei `fertig` gehören nicht zum untersuchten Ablauf.

**Signale in Ripes ablesen.** Fahren Sie mit der Maus über einen Anschluss: Ein Tooltip zeigt Name und Wert. *Display signal values* blendet alle Werte ein, Rechtsklick auf einen Anschluss und *Show value* einen einzelnen. Ein Multiplexer markiert den gewählten Eingang mit einem grünen Punkt; 1-Bit-Leitungen sind bei 1 grün, bei 0 grau. Alle Steuersignale kommen aus der Komponente `control`:

| Mini-Modell | Ausgang von `control` | geht an | Werte in Ripes |
|---|---|---|---|
| RegWrite | `reg_do_write_ctrl` | `registerFile`, `wr_en` | 0 / 1 |
| MemWrite | `mem_do_write_ctrl` | `data_mem`, `wr_en` | 0 / 1 |
| MemRead | `mem_ctrl` | `data_mem`, `op` | Zugriffsart LW, SW oder NOP |
| ALUSrc | `alu_op2_ctrl` | MUX `alu_op2_src` | 0 = Register, 1 = Immediate |
| MemtoReg | `reg_wr_src_ctrl` | MUX `reg_wr_src` | 0 = Speicher, 1 = ALU, 2 = PC + 4 |
| ALUSub | `alu_ctrl` | `alu`, `ctrl` | Operation, z. B. ADD |
| Branch | `do_branch` | UND-Gatter `br_and` | 0 / 1 |

Drei Abweichungen vom Mini-Modell sollten Sie kennen:

- Der Rückschreib-MUX ist umgekehrt codiert und hat einen dritten Eingang für PC + 4.
- Bei `beq` berechnet die ALU das Sprungziel PC + Immediate. ALU-Eingang B steht deshalb auf Immediate, und `alu_ctrl` zeigt ADD. Den Vergleich übernimmt die Komponente `branch`; ihr Ausgang `res` entspricht Zero.
- Der Datenspeicher hat kein eigenes Lese-Bit; `op` zeigt die Zugriffsart.

Die Namen gelten für aktuelle Ripes-Versionen. Notieren Sie Abweichungen Ihrer Version im Protokoll.

#### Musterlösung

Erwartung bei `fertig`: t1 = 12, t2 = 30, t3 = 42, t4 = 42, t5 = 42; Wörter ab B: 12, 30, 42, 42. Der Branch `beq t3, t4, treffer` wird genommen.

Mit Sollwert 41 wird t4 = 41. Der Branch ist nicht genommen, das Programm läuft über `kein_treffer` und setzt t5 = 0. Wörter ab B: 12, 30, 42, 41.

Erwartete Ripes-Werte laut Ripes-Quellcode, Spalten in der Reihenfolge der Zuordnungstabelle:

| Befehl | RegWrite | MemWrite | `mem_ctrl` | ALU-Eingang B | Rückschreib-MUX | Branch |
|---|---|---|---|---|---|---|
| lw | 1 | 0 | LW | 1 (Imm.) | 0 (Speicher) | 0 |
| add | 1 | 0 | NOP | 0 (Reg.) | 1 (ALU) | 0 |
| sw | 0 | 1 | SW | 1 (Imm.) | 1 (ALU) | 0 |
| beq | 0 | 0 | NOP | 1 (Imm.) | 1 (ALU) | 1 |

Abweichend von der Tabelle aus Aufgabe 2 sind nur die Codierung des Rückschreib-MUX sowie bei beq die Immediate-Auswahl und ADD statt Subtraktion, weil die Ripes-ALU dort das Sprungziel bildet.

Beim Store ist RegWrite null. Der gewählte Rückschreibwert wird nicht übernommen; die Store-Daten kommen aus RD2. Ripes wählt dort konkret das ALU-Ergebnis (`reg_wr_src` = 1), das X der Tabelle erlaubt beide Werte. Das sind Modellvorhersagen; Portnamen und Codierungen der eigenen Ripes-Version sind im Protokoll festzuhalten.